



Technische
Universität
Braunschweig

**Decision
Support**

Institut für Wirtschaftsinformatik



Data Driven Decision Making

Lecture 7 – Look-ahead Methods

Policy Classes (Compare Powell 2011)

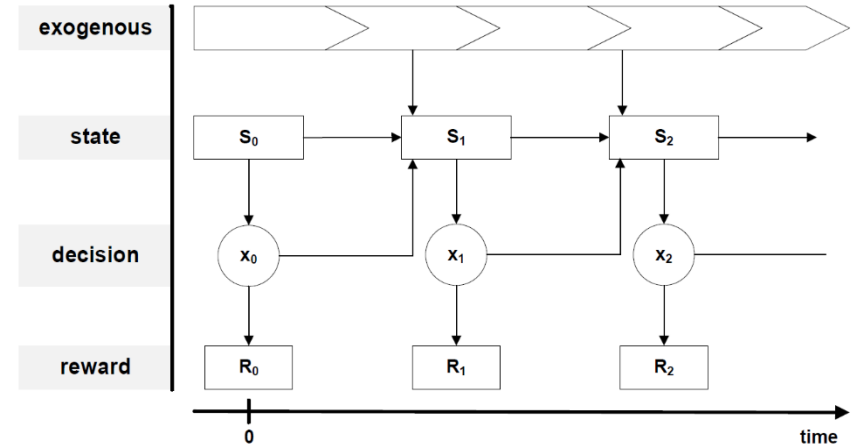
- Policy Function Approximation
- Rolling Horizon Re-Optimization
- Cost Function Approximation
- Look-ahead Models
- Value Function Approximation

Rule-Based

Optimization-Based

Policy Classes

- Policy function approximation:
Transfer a rule-of-thumb into a policy
- Rolling-Horizon Re-Optimization:
Determine decision based on current information (myopic)
- Cost-Function Approximation:
Manipulate reward function or constraints to incentivize (/avoid) (in)flexible decisions
- None of the classes anticipates the future (information & decision model) explicitly



Agenda

- **Multiple Scenario Approach**
- Post-Decision State Rollout

Sampling + Mixed Integer Programming

- Idea: Instead of state transitions, sample stream of stochastic information (for example, customer requests, production orders, demand, etc.)
- Extend state information with sampled information
- Solve mixed-integer program (maximize reward)
- Remove parts of solution that consider sampled information
- Implement (partial) solution
- Advantage: Conventional MIP-solvers can be used
- Disadvantage: Only one sample used for decision making (overfitting)

Example: Production Planning

State



Sampling



Solve



Implement



Multiple Scenario Approach (MSA)

- Bent, Van Hentenryck 2004
- Two-stage stochastic programming “light”
- Sample set of scenarios, determine solution that fits all scenarios best
- Sample several (streams of) information model realizations (e.g., sets of requests)
- Solve deterministic MIPs individually
- Clean solutions from augmented information (and repair, if necessary)
- Compare solutions and select “most similar” solution
- Advantage: Conventional MIP-solver can be used
- Disadvantage: stochasticity only captured implicitly in solution selection, no dynamism

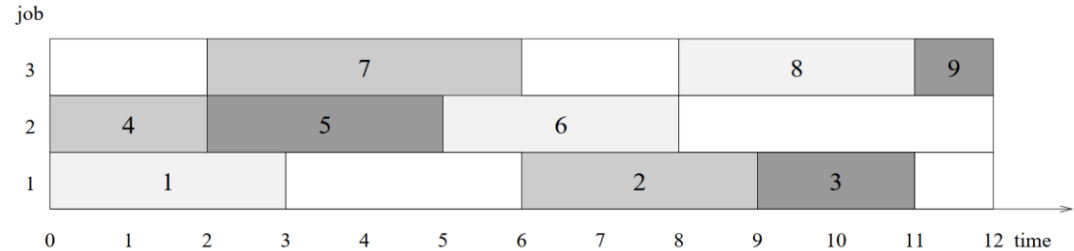
MSA: Consensus Function

- The consensus function determines the “most similar” solution
- Nice idea, but highly problem dependent
- Consensus function for production planning:
 - Solution: Sequence of jobs per production line
 - Consider only next job each line produces
 - Similarity measure between two solutions: number of identical jobs
 - Select solution with maximum similarity
- Example: 3 lines, A, B, C, six jobs 1,2,3,4,5,6
 - Solution 1: A**1**2, B**3**4, C56
 - Solution 2: A**1**3, B45, C26
 - Solution 3: A21, B**3**5, C46
- Solution 1 is the most representative solution and (partially) implemented

Excursion: consensus for schedules

C Bierwirth, DC Mattfeld, H Kopfer: On permutation representations for scheduling problems, International conference on parallel problem solving from Nature, 1996

- In a job shop, three jobs (1,2,3), (4,5,6) and (7,8,9) are to be processed by three machines
- The three machines are depicted in light grey, grey and dark grey.
- We aim at measuring similarity among schedules. Therefore we consider precedence.
- This schedule is represented as binary vector, where operations to be processed on the same machine are depicted in terms of preference relations.
- Operator prior (<) results in bitstring
- The schedule reads as follows:
- The hamming distance $D_{x,y} = \sum_i xor(x_i, y_i)$ measures similarity between two schedules
- Consensus: Pick the schedule with smallest sum of distances to all other schedules



(2<4)(2<7)(4<7)(1<6)(1<8)(6<8)(3<5)(3<9)(5<9)

0 0 1 1 1 1 0 1 1

Excursion: consensus for facility location / travelling salesman

- The facility location problem picks a subset of locations from a potentially larger set of possible locations

- Representation of solution:

1	2	3	4	5	6	7	8
0	1	1	0	1	0	1	0

- The symmetric travelling salesman problem constructs a tour among all locations by selecting arcs that connect nodes

- Representation of solution:

2/3	2/7	2/5	3/5	3/7	5/7
1	1	0	1	0	1

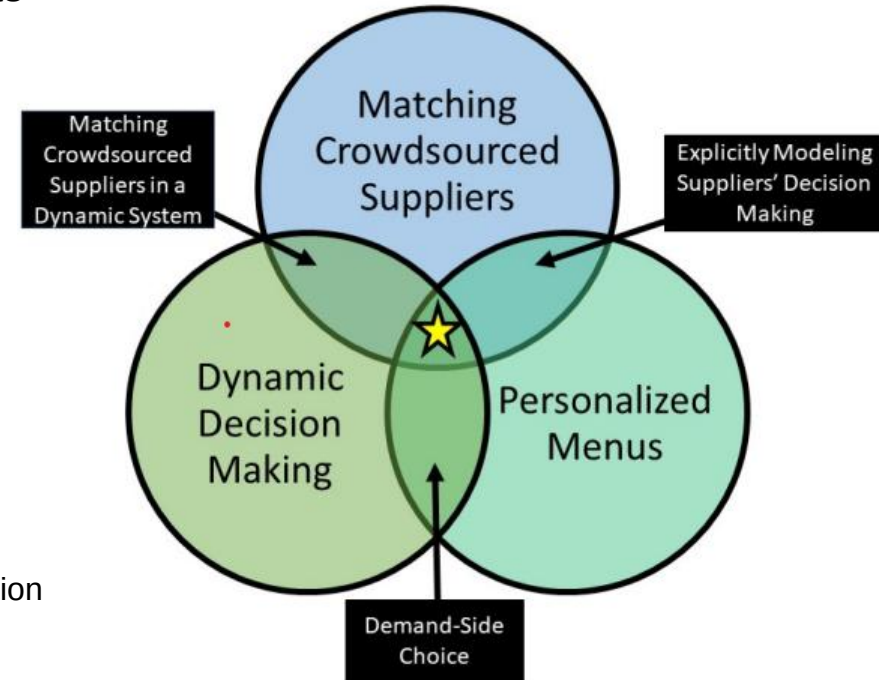
- Consensus reflects either precedence, position or adjacency



Case study: Autonomy of Crowdsourced Suppliers

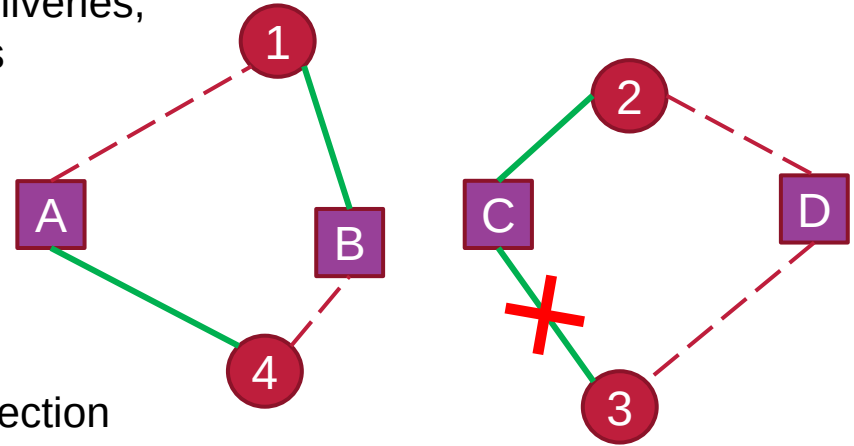
- In a gig-economy, suppliers work for multiple clients
- Suppliers therefore can reject offered assignments
- Reasons: travel duration, location, payment, etc.
- Problems arise for the requesting client
 - Less money for the platform
 - Delays, customer cancelations
 - Frustration among drivers who arrive late
- Hedge against rejections with so called menus

Ausseil, Rosemonde, Jennifer A. Pazour, and Marlin W. Ulmer.
"Supplier menus for dynamic matching in peer-to-peer transportation platforms." *Transportation Science* 56.5 (2022): 1304-1326.



Menus

- For every supplier a menu of potential customers is offered
- Instead of asking suppliers about favorite deliveries, a choice model depicts possible preferences
- Every supplier selects her/his “best” choice from the menu of suggested deliveries
- Platform finally decides on assignment
- Multiple offers increase the probability of selection
- Duplicate selection cause frustration of supplier
- How to find the right balance?

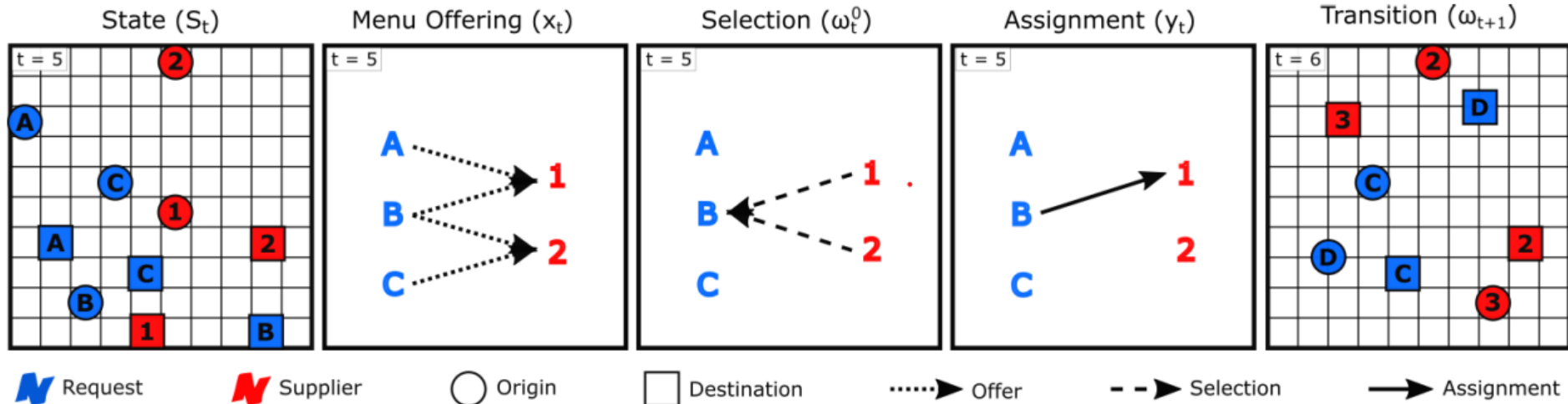


 Supplier

 Customer

Decision sequence

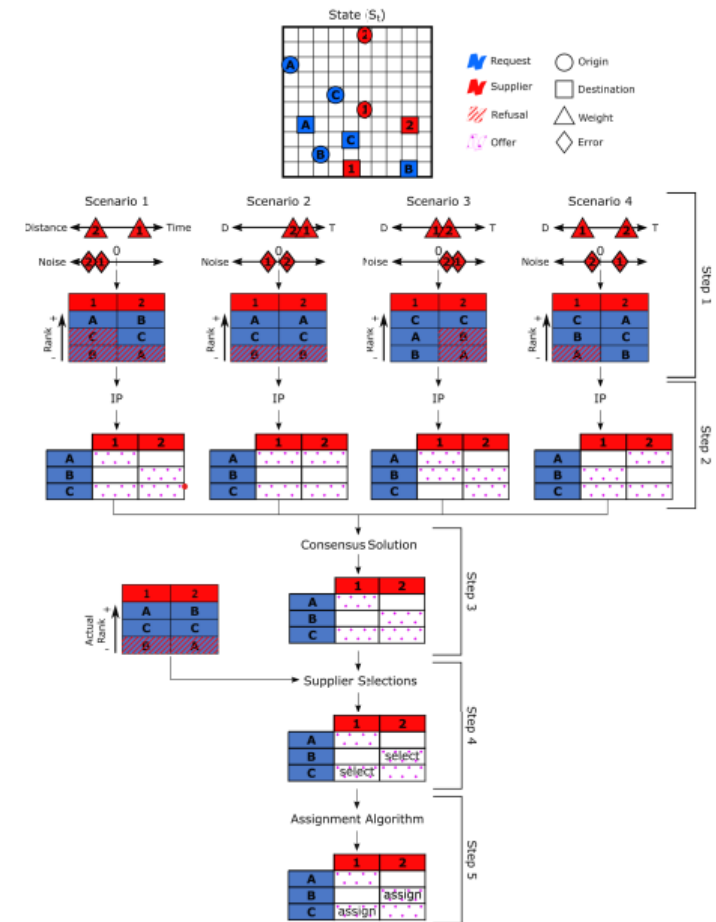
- At time $t=5$ two suppliers and three request are observed
- Request A and B are offered to supplier 1, B and C are offered to supplier 2
- Both suppliers opt for request B, and eventually B is assigned to supplier 1 $\rightarrow$ implement
- At time $t=6$ A has disappeared, C remains, D is new and supplier 3 has come into play



Multiple Scenario Approach

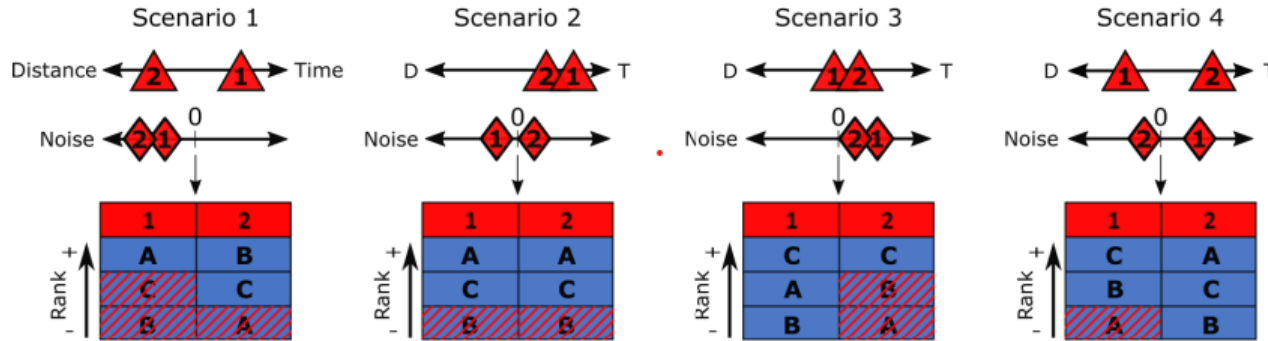
In each state

- Generate a set of scenarios
- Scenario is a known ranking of customers for each supplier
- Solve individual scenarios with some IP-magic
- Derive “consensus” solution from scenario solutions
- Observe customer selection of supplier
- Perform final assignment of customers to supplier



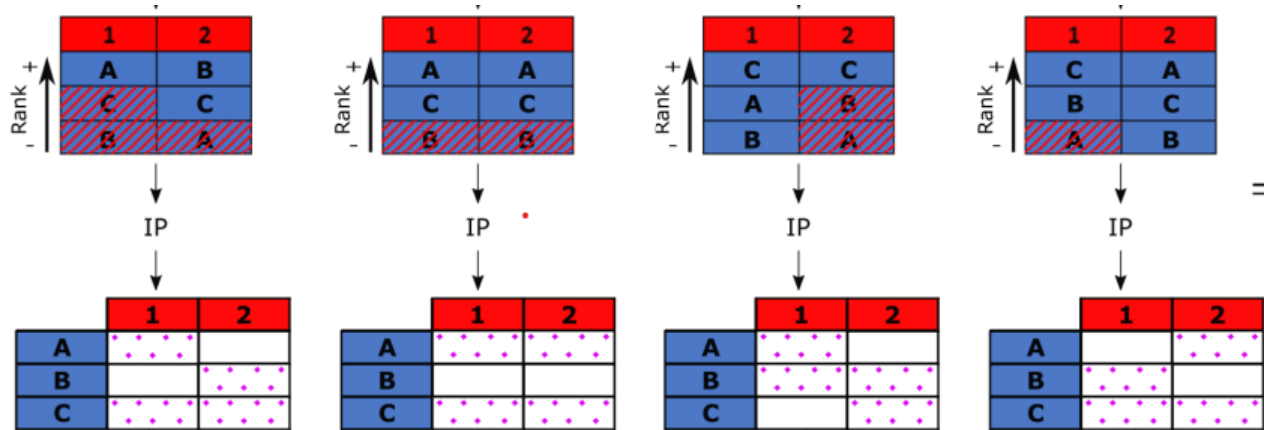
Step 1: Generating scenarios

- Preferences of suppliers with regard to attributes of the current state are not known
- Distance to the customer and travel time are identified as meaningful attributes
- The weights of attributes are modified for each supplier at random
- Solving the respective MIP leads to a ranking/rejection of requests for suppliers



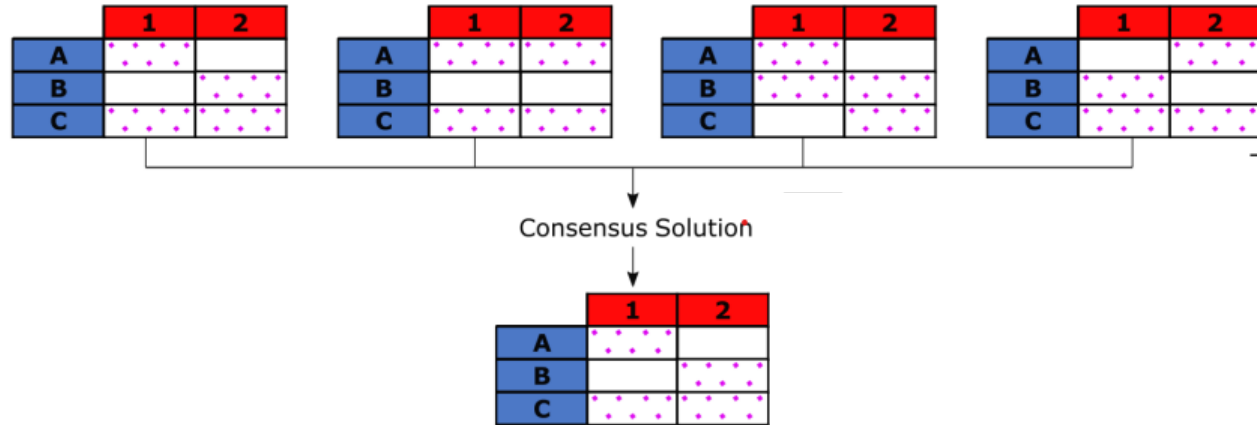
Step 2: Solving an bi-level Integer Program

- First level: Assignment of customer requests to supplier in a scenario
- Second level: Supplier (artificially) select requests in accordance to rank
- Conversion of second level objective function in constraint $\rightarrow$ single-level IP
- Solve the resulting assignment problem for $x_{ijt\sigma}$ supplier i , customer j , state t , scenario σ
- The assignment problem is variant of the transportation problem $\rightarrow$ modi method



Step 3: Consensus solution

- Reflects the structure discovered by the scenario solutions by means of a majority vote
- Compute frequency $f_{ijt} = \sum_{\sigma} x_{ijt\sigma}$ over scenarios σ and maximize $\sum_i \sum_j f_{ijt} x_{ijt}$
- The consensus x_{ijt} considers frequently observed requests in one feasible solution

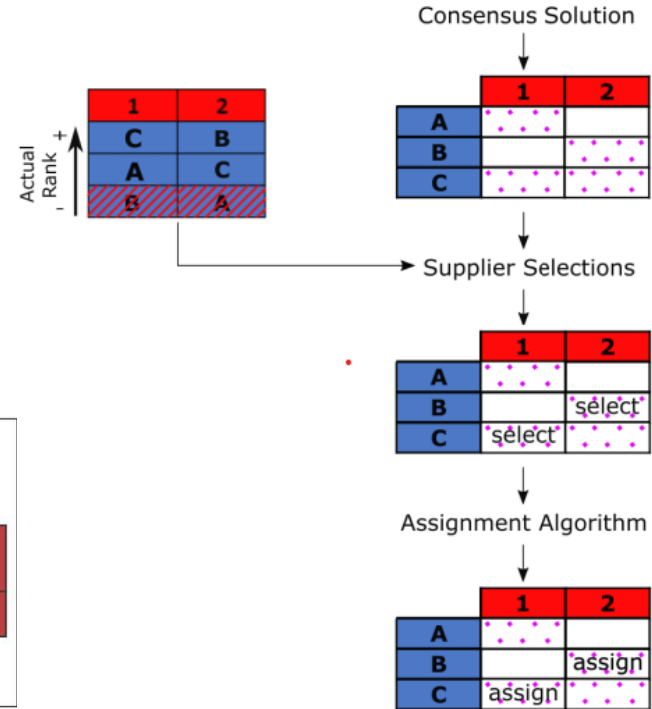
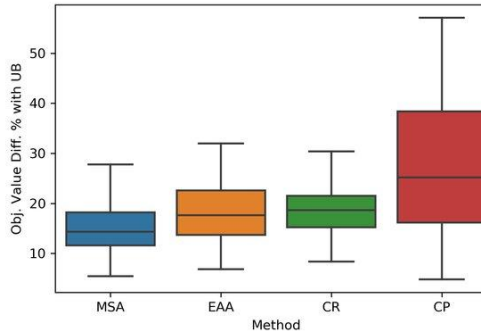


Step 4 and 5: Supplier selection and final assignment

- The consensus solution is transferred to all suppliers
- The suppliers individually select favorable requests
- Finally, a non-overlapping assignment is derived that is hopefully acknowledged by the suppliers

Results:

- MSA: Our Approach
- EAA: mean value approach
- CR: Offer n closest customers
- CP: Current practice
- UB: Upper Bound



Summary

- Menus can reduce waiting times for customers and suppliers
- Menus are particularly important if supplier behavior is uncertain
- In practice, multiple attributes describing individual drivers are taken into account
- Drawback: requests unfavorable to all suppliers have to be assigned as well
- Frustration → ask for a request selection first and assign other requests later on?
- This matter is subject to investigation in lecture 11 → food delivery

MSA Summary

- Multiple scenario approach considers the entire decision space.
- MSA does not capture dynamism explicitly.
- Decisions are derived at once and not subsequently.
- Controllable stochasticity:
 - Time windows
 - Start times, etc.
 - Capture dynamism implicitly
- Static solutions → always on the edge of infeasibility
- Alleviated by the “right” consensus function

Agenda

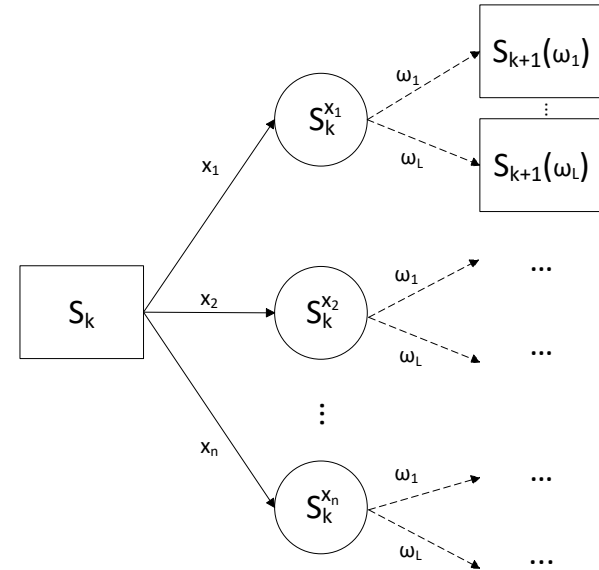
- Multiple Scenario Approach
- **Post-Decision State Rollout**

Look-ahead Methods

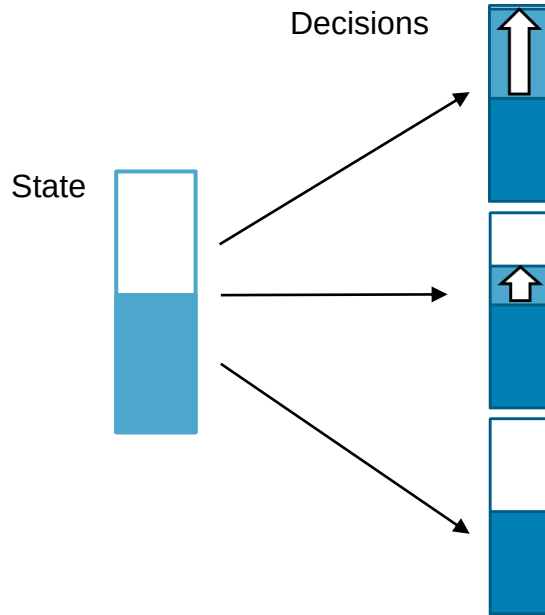
- Idea: Build and solve part of the MDP
- Components
 - Information model: Sampling from information model distribution
 - Decision model: None, Mixed Integer Program or given Base Policy
 - Decision Points: Limited look-ahead Horizon
- Many different look-ahead methods
 - Sampling
 - Sampling + Rollout of policy

Sampling

- State S_k and set of decisions $X(S_k)$
- Post-decision state candidates $S_k^{x_1}, S_k^{x_2}, S_k^{x_3}, \dots$
- For every post-decision state S_k^x :
 - Sample set of information model realizations
 - Use realizations to evaluate decisions (for example, incurred costs, etc.)



Example: Inventory Problem



Samples

$$\frac{1}{L} \sum_{l=1}^L R(S^{x_1}, \omega_l)$$

$$\frac{1}{L} \sum_{l=1}^L R(S^{x_2}, \omega_l)$$

$$\frac{1}{L} \sum_{l=1}^L R(S^{x_3}, \omega_l)$$

Samples may be generated independently of decision x and tested with hindsight

Suitable Application for Sampling Transitions

- Approximation of reward function (demand, travel time, etc.)
- Short-term anticipation of stochastic information
- Very dynamic problems (short lookahead horizon)
 - Emergency services
 - Dial-a-ride
- To evaluate decisions that are not impacted much by future decisions
 - Inventory routing in bike sharing systems
 - A station is served usually only a few times within the day

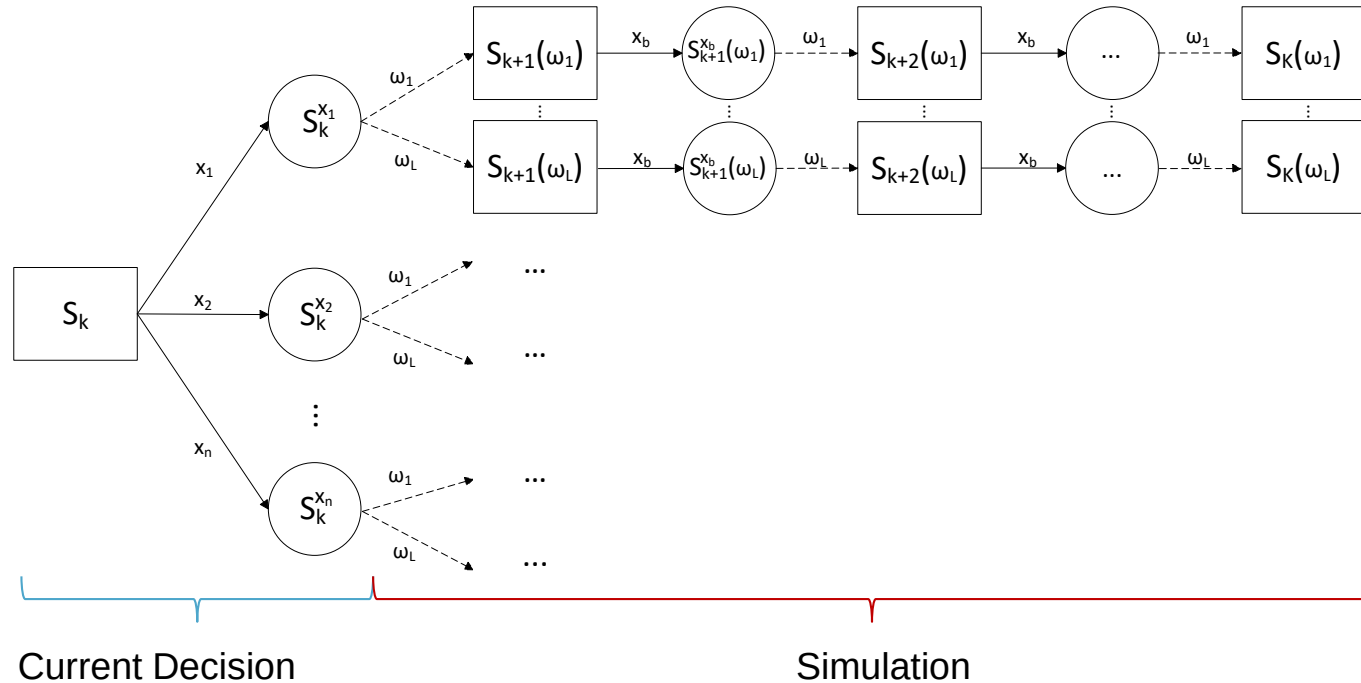
Post-Decision Rollout Algorithm (Bertsekas, 1995)

- State S_k and set of decisions $X(S_k)$
- Post-decision state candidates $S_k^{x_1}, S_k^{x_2}, S_k^{x_3}, \dots$
- For every post-decision state S_k^x :
- Simulate L paths (information model realizations + decisions)
- Decisions within simulation: (runtime-efficient) base policy π_b
- Receive L results
- Define $\hat{V}(S_k^x)$ as average over simulated results

$$\hat{V}(S_k^x) = L^{-1} \sum_{l=1}^L \left[\sum_{i=k+1}^K R(S_i, X_i^{\pi_b}(S_i)) | S_k^x, \omega^l \right]$$

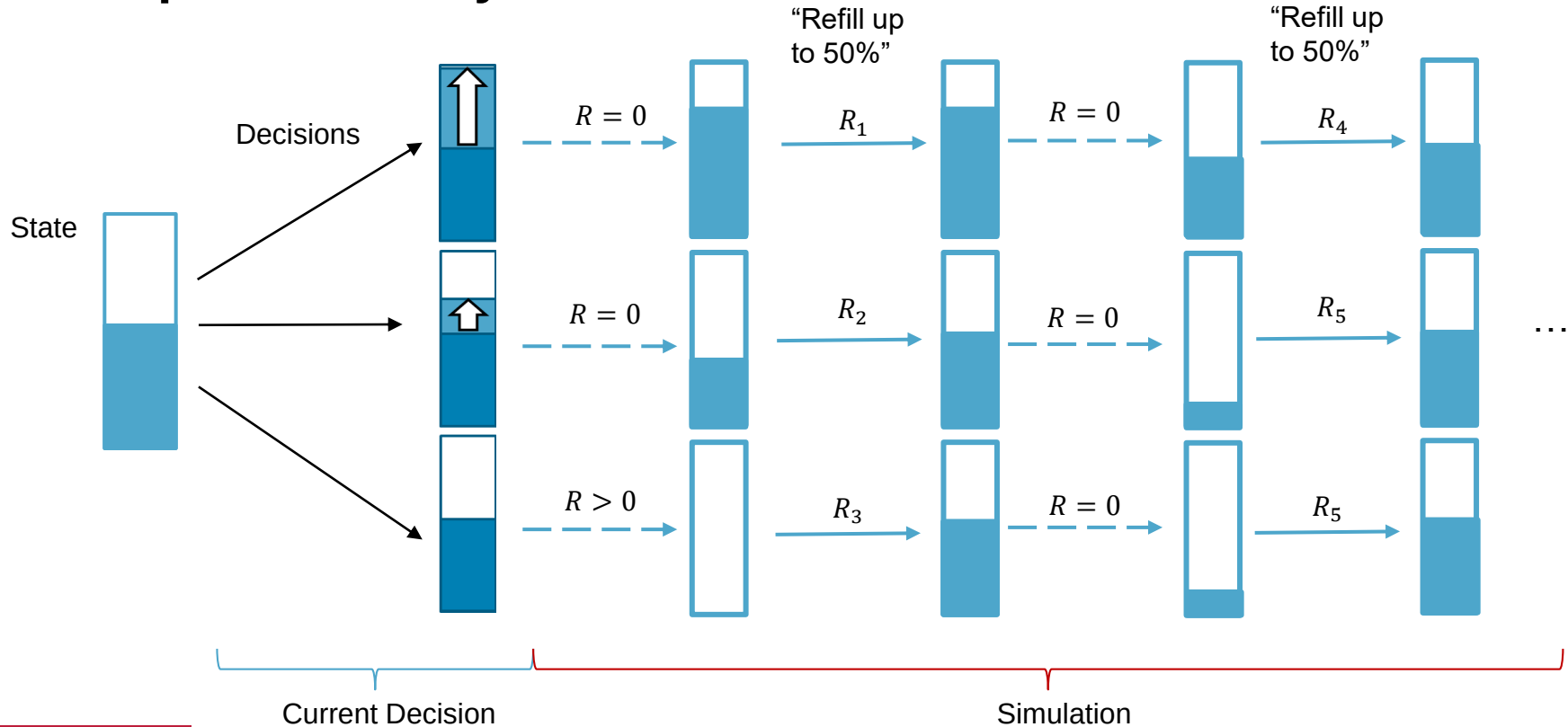
- Select decision x , such that $R(S_k, x) + \hat{V}(S_k^x)$ is maximized

Post-Decision Rollout Algorithm: Visualization



Example: Inventory Problem

$$X_k^{\pi^*}(S_k) = \arg \min_{x \in X(S_k)} \{R(S_k, x) + V(S_k^x)\}$$



Summary

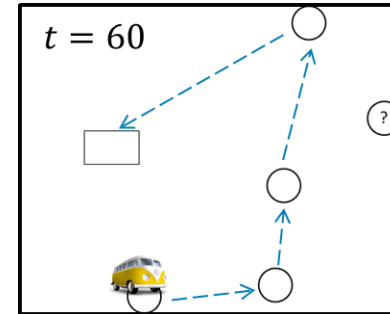
- Can capture dynamic development of information and decisions
- Advantageous, for example, to capture state developments over time (vehicle movements, job arrivals, etc.)
- Only a few decisions can be evaluated
- Decisions significantly impacted by base policy
- The longer the horizon, the further away from realistic developments
- Can be combined with runtime efficient PFAs or CFAs as base policies (and value function approximation, as we see in Lecture 10)

Agenda

- Look-ahead Methods
- **Case Study I: Post-Decision State Rollout**
- Case Study II: Multiple Scenario Approach

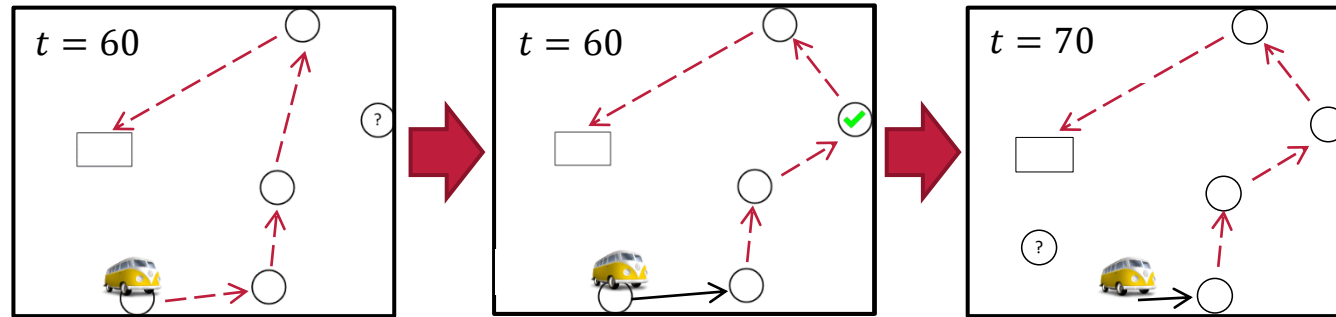
Case Study: Courier Service

- Single vehicle
- Depot (start and finish)
- Limited travel time (shift)
- Customer request pickup of parcels
- Part of the customers is known in advance
- Part of the customer request dynamically throughout the day
- Decisions:
 - Acceptance
 - Routing



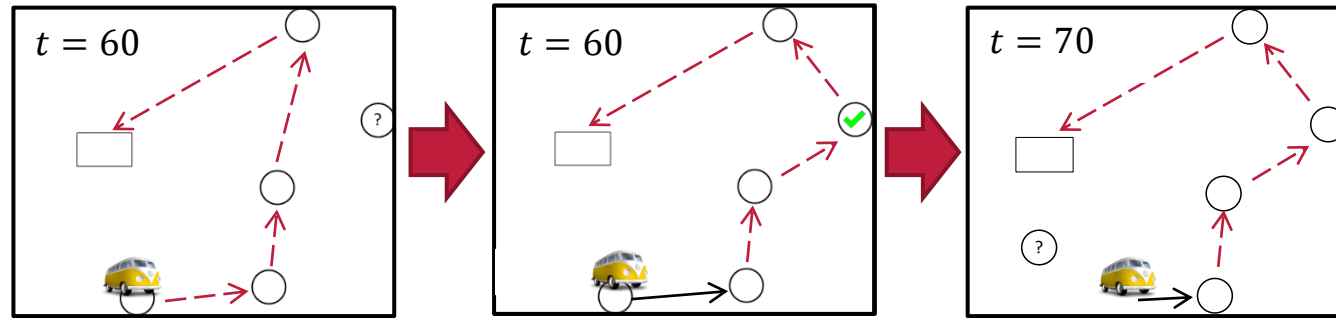
Goal: Maximize the expected number of accepted dynamic requests

Markov Decision Process



- Decision Point: New Request
- State: time, vehicle position, customers to serve, new request, planned tour
- Decisions: Acceptance/Rejection, Adaption of the tour
- Reward: 1 (acceptance) or 0 (rejection)
- Transition: Travel and new customer request

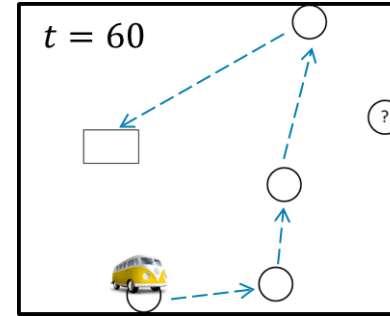
Rollout Algorithm: Decision Space Reduction



- Rollout Algorithm can evaluate only a limited number of decisions
- Acceptance: 2 decisions
- Routing: n customers in the tour, 1 request: Up to $(n+1)!$ Decisions
- Reduction:
 - Explicit decision on acceptance
 - Routing heuristic: Keep the current tour and insert in the “cheapest” position

Acceptance and the Value Function

- Acceptance of a request reduces the available resources to serve future requests. (value decreases)
- Question: Is it worth accepting a request?
 - Valuable
 - If resource reduction is low,
 - If, through resource investment, future requests are more efficient to use (coverage)
- Bellman Equation:

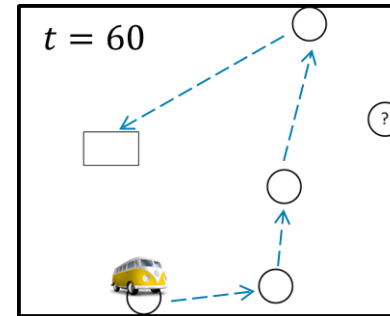


$$X_k^{\pi^*}(S_k) = \arg \max_{x \in X(S_k)} (R(S_k, x) + V(S_k^x))$$

Acceptance Decisions

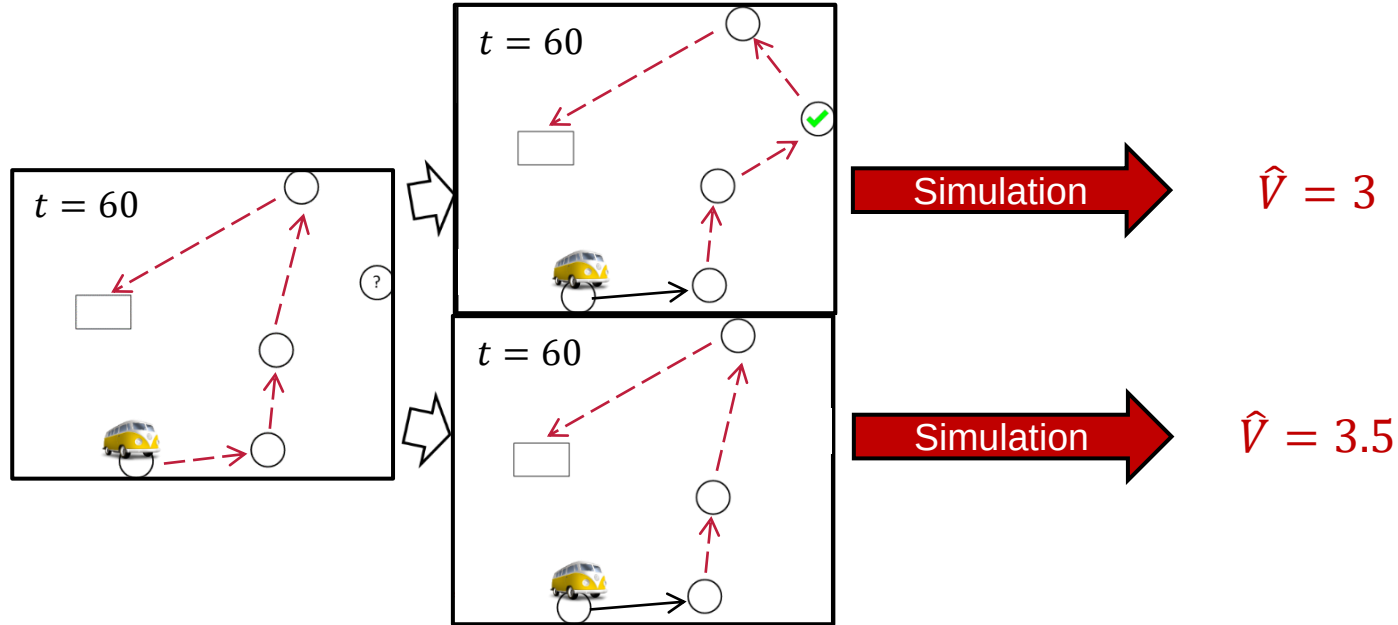
$$X_k^{\pi^*}(S_k) = \arg \max_{x \in X(S_k)} (R(S_k, x) + V(S_k^x))$$

- 2 Policies
- Myopic:
 - Value function is set to 0
 - Maximize immediate reward
 - Customers are always accepted as long as they can be served
- Rollout Algorithm:
 - For each of the two post-decision states
 - Simulate the rest of the day
 - Use myopic policy as a basic policy
 - Number of simulation runs $L=16$

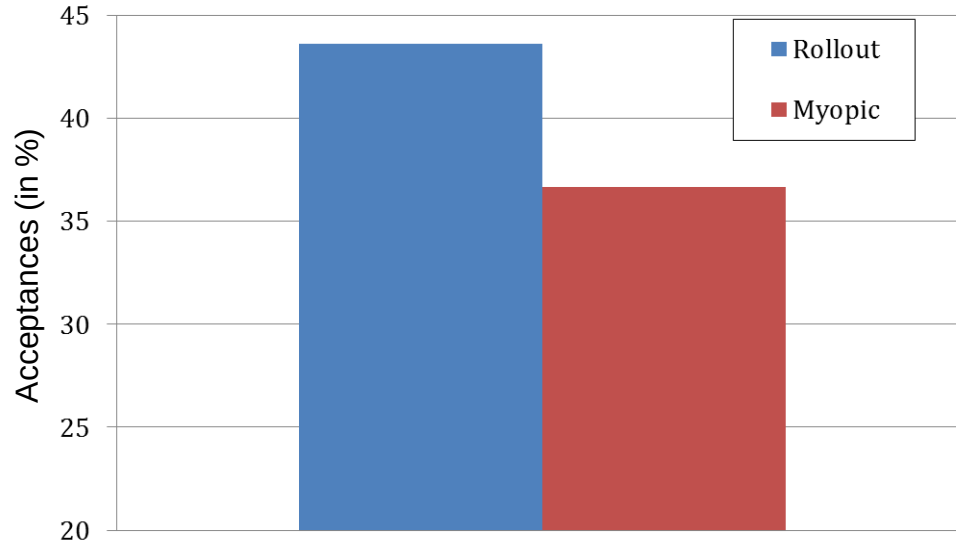


Example

$$X_k^{\pi^*}(S_k) = \arg \max_{x \in X(S_k)} (R(S_k, x) + V(S_k^x))$$

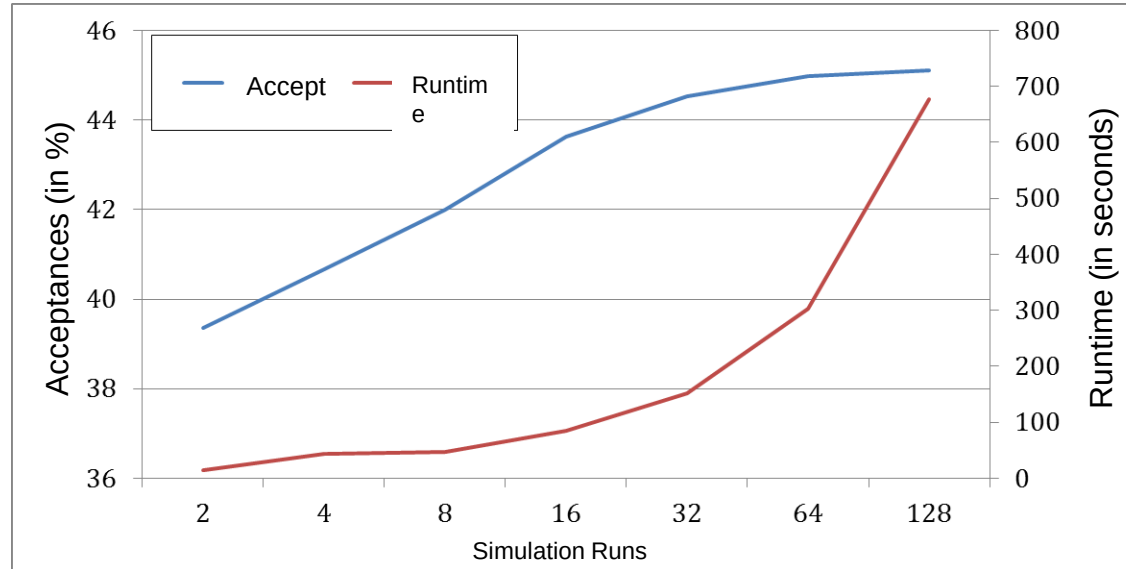


Results



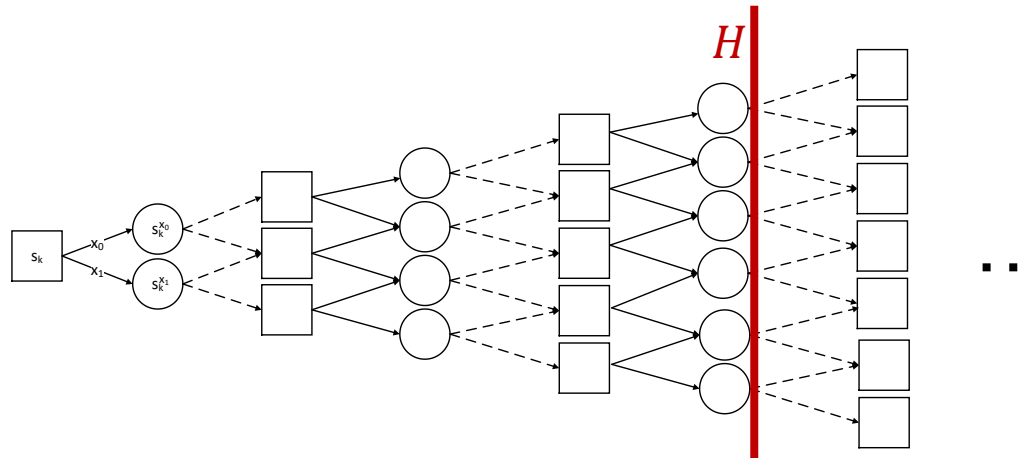
Tradeoff: Simulation runs vs. quality

- Runtime and solution quality increase with number of runs



Limiting the simulation horizon

- Idea: limiting the simulation according to H decision points
- Expected: tradeoff between anticipation and reliability
- The longer the simulation horizon, the more anticipation
- The shorter the simulation horizon, the more reliable the simulation



Limited Horizon

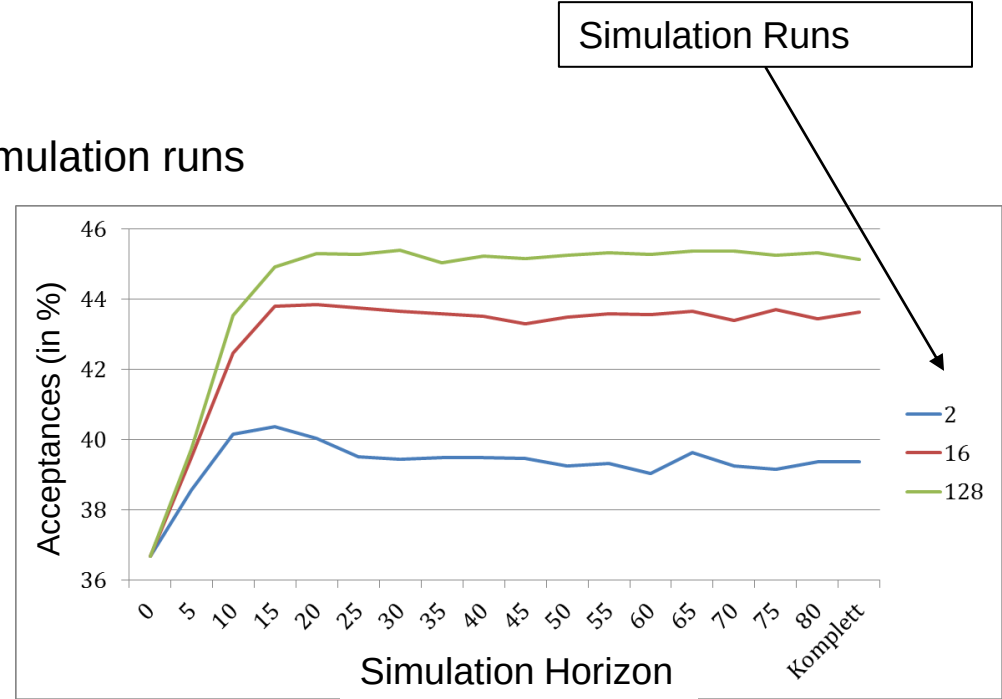
- Adaption of Bellman Equation:

$$\hat{V}(S_k^x) = L^{-1} \sum_{l=1}^L \left[\sum_{i=k+1}^{\min(K, k+H)} R(S_i, X_i^{\pi_b}(S_i)) | S_k^x, \omega^l \right]$$

- Simulation of maximally H decision points
- $H = 0$: myopic (fast), $H = \infty$: full simulation to the final state (slow)
- Alternative (not considered here): Artificially discounting rewards by
- $(1 - \gamma)^{i-(k+1)}$ with discount factor γ , may require more runtime.

Simulation Horizon

- Results for $L = 2, 16, 128$ and $H = 0, 5, 10, \dots, 80, \infty$
- Initial increase in solution quality
- Slight decrease for longer horizons
- Behavior depends on the number of simulation runs
- (decrease in runtime for small H)



Summary

- MDP are often too complex to solve exactly
- Idea: Use of online simulation to estimate the value function

$$V(S_k^x) = \mathbb{E} \left[\sum_{i=k+1}^K R(S_i, X_i^{\pi^*}(S_i)) | S_k^x \right] \qquad \hat{V}(S_k^x) = L^{-1} \sum_{l=1}^L \left[\sum_{i=k+1}^{\min(K, k+H)} R(S_i, X_i^{\pi^b}(S_i)) | S_k^x, \omega^l \right]$$

- Simulation is based on realizations and a basic policy
- Disadvantages of Rollout Algorithm
 - Runtime is limited
 - Only a few decisions can be considered
 - Base policy leads to suboptimal decisions in simulation

Look-ahead: Advantages and Disadvantages

- Advantages:
 - Full state information is considered
 - Stochasticity is considered explicitly
 - Full decision space XOR
 - Dynamic developments can be considered
- Disadvantages:
 - Runtime
 - Information model: only a few realizations can be considered
 - Decision model: only a few decisions can be considered in simulation
 - Divergence from realistic developments with increasing horizon.
 - Conventional solvers may not be suitable, e.g., for rollout algorithms

Back to the Lecture: Outlook

- Can we store simulation results?
- Machine Learning
- Lecture 8: Reinforcement Learning
- Lecture 9: Analytics Issues in RL